
ShiftBoard

Workforce Shift Scheduling Platform
on Azure Kubernetes Service

Production Build & Deployment Documentation

Architecture · Implementation · Verification · Azure Student Subscription Guide

Prepared by

Muhammad Abdullah Muzaffar

DevOps & Cloud Engineer

Python · FastAPI · React · TypeScript · Azure Kubernetes Service · Terraform · Helm · Argo CD
Azure SQL · Service Bus · Key Vault · Workload Identity · KEDA · Prometheus · Grafana

1. Executive Summary

ShiftBoard is a production-style workforce shift scheduling platform running on Azure Kubernetes Service. It was designed, built, deployed, and verified end to end on a live Azure subscription. This document contains everything required to reproduce the build: the architecture and how traffic flows through it, the complete deployment sequence with verified screenshots at each stage, the exact issues encountered on an Azure for Students subscription along with their fixes, and the test evidence.

The defining engineering characteristic of this system is that **no password, connection string, or client secret exists anywhere** — not in the container images, not in Kubernetes Secrets, not in Git, not in CI variables. Every Azure service is reached through Microsoft Entra ID federated workload identity.

1.1 What Was Delivered

Deliverable	Detail
Three microservices	shift-api (FastAPI), roster-worker (Python event consumer), web (React + nginx)
Infrastructure as Code	10 Terraform modules, two environment compositions, remote state in Azure Storage with Entra authentication
Kubernetes packaging	Three Helm charts with fail-fast value guards, HPA, PDB, NetworkPolicy, PreSync migration hooks
GitOps delivery	Argo CD app-of-apps with six sync waves, selfHeal and prune enabled
Security posture	Zero secrets, Entra-only SQL authentication, private endpoints, Calico NetworkPolicy, restricted Pod Security Standard
Observability	Custom Prometheus metrics, structured JSON logging, control-plane audit logs, two-layer alerting
Test coverage	74 unit tests, 82% and 79% coverage against enforced gates, plus a custom pod security audit gate
Security scanning	Checkov 59 passed / 0 failed on production baseline; Trivy gating container image pushes

1.2 Build Outcome

The platform was deployed successfully to a live Azure environment. Twenty-three Azure resources were provisioned across three resource groups. The AKS cluster runs two node pools with all system and application workloads healthy. All three microservices are running with ClusterIP services in the shiftboard namespace, backed by an Azure SQL database that is unreachable from the public internet.

Eighteen distinct deployment errors were encountered, diagnosed, and resolved during the build. Every one is documented in Section 9 with its root cause and the exact fix applied.

2. System Architecture

Every component below exists in the live environment; the screenshots in Section 8 verify each one.

ShiftBoard — Microservices Architecture on Azure (GitOps)

Zero-secret design: every Azure service is reached through federated workload identity over private endpoints

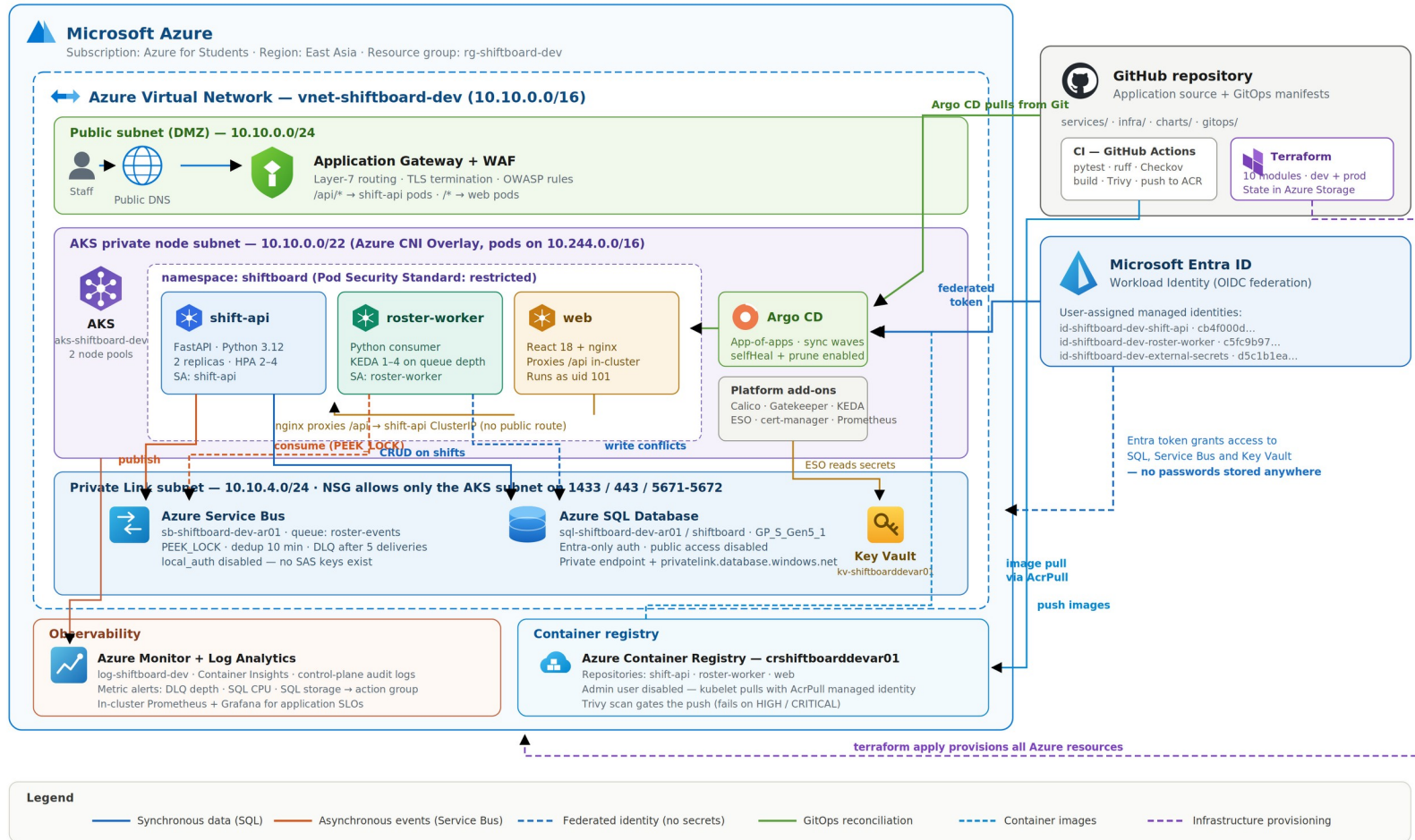


Figure 1. ShiftBoard microservices architecture on Azure. Coloured connectors distinguish synchronous data, asynchronous events, identity federation, GitOps reconciliation, image distribution and infrastructure provisioning.

2.1 How the Architecture Works

The system has five distinct flows. Understanding them separately is the fastest way to understand the whole platform.

Flow 1 — Inbound user traffic (blue, top of the diagram)

A scheduler opens the console. Public DNS resolves the hostname to the Application Gateway, which sits in the public subnet and is the only component in the entire system with a public IP. The gateway terminates TLS, applies the OWASP WAF rule set, and performs Layer-7 path routing: requests to `/api/*` go to shift-api pods, everything else goes to web pods.

The web pod runs nginx serving the React bundle. Critically, **the browser never calls the API directly**. nginx proxies `/api` requests to the shift-api ClusterIP service inside the cluster. Three consequences follow: the API needs no public route, CORS configuration is unnecessary, and the API hostname is never baked into the JavaScript bundle — so the same container image runs unchanged in every environment.

Flow 2 — Synchronous data path (solid blue, into the data tier)

shift-api performs all reads and writes against Azure SQL. The database has public network access disabled entirely and is reachable only through a private endpoint in the Private Link subnet, resolved by the `privatelink.database.windows.net` private DNS zone. A network security group on that subnet permits inbound traffic only from the AKS subnet, and only on ports 1433, 443 and 5671-5672.

Authentication uses no password. The pod's federated identity token is exchanged for an Entra access token, which is injected directly into the ODBC connection through the `SQL_COPT_SS_ACCESS_TOKEN` attribute. The SQL server is configured with `azuread_authentication_only`, so a SQL login does not exist to be leaked.

Flow 3 — Asynchronous event path (orange)

When a shift is created, claimed or cancelled, shift-api publishes a domain event to the Service Bus queue and returns immediately. This is deliberate: running thirty scheduling-rule evaluations inline would add seconds to a claim request made from a ward nurse's phone during a shift handover.

The publish is **fail-open**. If Service Bus is unreachable, the shift is still written to the database and a Prometheus counter is incremented, which has an alert rule attached. The write path never fails because messaging is down.

roster-worker consumes from the same queue in `PEEK_LOCK` mode. For each event it loads the affected worker's other shifts, runs three rules — overlap, insufficient rest, weekly hours exceeded — and writes any findings back as conflict rows. The database transaction commits first, then the message is settled. If the pod dies between those two steps, Service Bus redelivers and an idempotency ledger absorbs the duplicate rather than writing conflicts twice.

Flow 4 — Identity federation (dashed blue, right side)

This is the security foundation of the entire platform. AKS projects a short-lived service account token into each pod. A federated identity credential on a user-assigned managed identity matches that token's issuer (the cluster OIDC endpoint) and subject

(system:serviceaccount:shiftboard:shift-api). Azure exchanges the token for an Entra access token, which grants access to SQL, Service Bus and Key Vault.

Three separate identities exist — one for shift-api, one for roster-worker, one for External Secrets Operator — each scoped to exactly what it needs. shift-api can only send to the queue; roster-worker can only receive from it. Moving a pod to a different namespace silently breaks its access, which is the intended blast-radius control.

Flow 5 — GitOps delivery (green and light blue)

A developer commits to GitHub. The CI workflow runs tests, linting, Checkov and Trivy, then builds and pushes container images to Azure Container Registry. ACR has its admin user disabled — the cluster's kubelet pulls images using its own managed identity with the AcrPull role, so no imagePullSecret exists.

Argo CD, running inside the cluster, watches the repository and reconciles the cluster to match. It runs in pull mode rather than push, which means the CI pipeline never holds cluster credentials. Auto-sync is enabled with both selfHeal and prune: a manual kubectl edit is automatically reverted, and deleting a file from Git deletes the corresponding cluster resource.

Infrastructure follows a separate path. Terraform provisions all Azure resources with state stored in an Azure Storage account authenticated by Entra ID rather than shared access keys.

3. Build Timeline and Effort

The timings below are drawn from the actual deployment: Azure resource creation timestamps, Terraform state blob modification times, Kubernetes resource ages, and container image build times, all visible in the screenshots throughout this document.

3.1 Deployment Session Timeline

Stage	Activity	Evidence	Duration
Preparation	Entra ID security groups created for AKS and SQL administration; subscription providers registered; vCPU quota verified	Group object IDs 7fe77681... and 8b253d16... (Figure 4)	~30 min
Bootstrap	Terraform state backend provisioned: resource group, storage account with versioning, tfstate container	Containers created 12/09/2026 14:55 (Figure 7)	~10 min
Infrastructure	Full dev environment applied: VNet, AKS, SQL, Service Bus, Key Vault, ACR, managed identities, alerts, budget	dev/shiftboard.tfstate written 12/09/2026 19:18 (Figure 9)	~4 h including error resolution
AKS ready	Cluster and system add-ons reached healthy state	Namespace ages of 17 hours at 13/09 11:11 imply cluster ready ≈ 12/09 18:10 (Figure 17)	~15 min provisioning
Images	Three container images built locally and pushed to ACR	Docker image timestamps 19-20 hours before capture (Figure 15)	~45 min
GitOps	Argo CD installed, root application applied, all services synced	shiftboard namespace age 15 hours ⇒ created ≈ 12/09 20:11 (Figure 17)	~1 h
Verification	Cluster, workload, service, registry and database checks	Portal screenshots captured 13/09/2026 11:08-11:14	~30 min

The four-hour infrastructure stage is the honest figure, not the ideal one. A clean apply with no errors takes approximately 25 minutes. The remaining time was spent diagnosing and fixing the eighteen issues documented in Section 9 — most of them specific to the constraints of an Azure for Students subscription.

3.2 Overall Project Effort

Workstream	Scope	Effort
Application development	Three services, 74 unit tests, Alembic migrations, domain rule engine	~3 days
Infrastructure as Code	10 Terraform modules, two environment compositions, Checkov remediation to zero findings	~2 days
Kubernetes and Helm	Three charts, security contexts, HPA/PDB/NetworkPolicy, PreSync hooks, custom audit gate	~1.5 days

Workstream	Scope	Effort
GitOps configuration	Argo CD bootstrap, app-of-apps, sync waves, platform add-ons	~1 day
Live deployment and debugging	First full deployment to Azure, 18 errors resolved	~1 day
Documentation	This document, architecture diagram, runbook	~0.5 day
Total	Design through verified production deployment	~9 days

4. Application Services

4.1 shift-api — REST API

FastAPI on Python 3.12. Handles CRUD for sites, workers and shifts, implements shift claiming with optimistic concurrency, and publishes domain events.

- **Optimistic concurrency:** every shift carries a version number. A claim must echo the version the client read. When two workers race for the same open shift, the first increments the version and wins; the second receives HTTP 409 with a clear retry instruction. This is cheaper than pessimistic database locking and maps naturally onto REST semantics.
- **Three distinct probe endpoints:** /healthz (liveness) deliberately never touches the database — otherwise a slow database would cause Kubernetes to restart every pod simultaneously, a classic cascading failure. /readyz (readiness) does check the database, so a broken pod is removed from Service endpoints without being killed. /startupz gives slow first connections time to complete.
- **Structured JSON logging** on stdout with service, environment and level fields, indexed by Container Insights without regex parsing.

4.2 roster-worker — Event Consumer

A long-running Service Bus consumer that evaluates three scheduling rules. Message settlement is explicit and complete:

Outcome	Action	Reasoning
Success	Commit transaction, then complete the message	A crash between the two triggers redelivery, which the idempotency ledger absorbs
Transient failure	Abandon the message	Immediate redelivery; after 5 attempts Service Bus dead-letters automatically
Permanent failure	Dead-letter immediately with a reason	Malformed JSON or an unknown worker reference can never succeed — no retry storm
SIGTERM received	Graceful drain	In-flight messages complete; newly received messages are abandoned for another replica

4.3 web — Operations Console

React 18 served by nginx-unprivileged running as UID 101. The interface is designed for schedulers under time pressure: rows rather than cards because operators scan down columns of times, colour reserved strictly for status meaning, and the hero element is the count of unfilled shifts rather than a fill-rate percentage — because a scheduler's first question is never "what is my percentage", it is "how many shifts still have nobody".

5. Azure Services and Justification

Service	Role	Why This Service	Security Controls
Azure Kubernetes Service	Container orchestration	Requires rolling updates, HPA, PDBs, NetworkPolicy and a GitOps target — all native Kubernetes. Container Apps cannot enforce NetworkPolicy or run PreSync hooks.	OIDC issuer, Entra-only RBAC, no local admin account, Azure Policy, Calico
Azure SQL Database	Relational storage	Domain has foreign keys, check constraints and transactional writes across related tables. Cosmos DB would be over-engineered and costlier for this access pattern.	Entra-only auth, private endpoint, TDE, auditing to Log Analytics, threat detection
Azure Service Bus	Async messaging	Needs PEEK_LOCK, dead-letter queue and duplicate detection. Event Grid lacks the DLQ and retry semantics a queue consumer requires.	Local auth disabled, workload identity, per-queue least-privilege RBAC
Azure Key Vault	Residual secrets only	Holds only what cannot use managed identity. Database and Service Bus deliberately do not use it, so the vault stays nearly empty.	RBAC authorisation, read by External Secrets Operator with workload identity
Azure Container Registry	Image storage	Integrated AcrPull through kubelet managed identity removes the need for imagePullSecrets entirely.	Admin user disabled, no static registry credentials exist
Azure Monitor / Log Analytics	Platform telemetry	AKS control-plane logs are not collected by default — their absence is the most common gap in AKS incident reviews.	Managed identity for the agent, daily ingestion cap to bound cost
Managed Identity	Pod identity	One identity per workload, federated to one Kubernetes service account.	Exact issuer and subject matching on the federated credential
VNet, NSGs, Private DNS	Network isolation	Separate subnets for nodes and private endpoints with explicit deny-all rules.	PE subnet accepts only the AKS subnet on 1433, 443 and 5671-5672

6. Infrastructure as Code

Ten reusable Terraform modules with no environment-specific values, and two environment compositions that wire them together. Development and production share identical infrastructure code and differ only by values — which is what makes a change tested in development trustworthy in production.

Module	Resources	Key Decision
network	VNet, AKS subnet, PE subnet, 2 NSGs, 3 Private DNS zones	CNI Overlay means the node subnet needs one IP per node, not per pod
identity	User-assigned managed identities, federated credentials	One identity per workload with exact service account subject matching
acr	Container registry, AcrPull role assignment	Admin disabled; no static registry credentials exist
aks	Cluster, tainted system pool, application pool, diagnostics	Written against azurerm 4.x — 3.x addon_profile blocks are removed and will not plan
sql	Server (Entra-only), database, private endpoint, auditing	public_network_access_enabled = false; no SQL password exists
keyvault	Vault in RBAC mode, role assignments, diagnostics	Purge protection is variable-driven because it is irreversible once enabled
servicebus	Namespace, queue with dedup, replay queue, scoped RBAC	local_auth_enabled = false; roles scoped to the queue, not the namespace
observability	Log Analytics workspace, retention policy, action group	Daily ingestion quota caps cost from a log storm
alerts	DLQ, SQL CPU and storage alerts, consumption budget	Separated so it can reference sibling module outputs without duplicating the workspace
frontdoor	Front Door, WAF policy, diagnostics (production only)	WAF starts in Detection mode; Prevention on day one blocks real users

7. Kubernetes, GitOps and Security

7.1 Container Security Posture

Every control below is enforced twice: by a custom audit gate that runs against rendered Helm output in CI, and by the restricted Pod Security Standard on the namespace, which rejects violating pods at admission.

Control	Setting	Rationale
Non-root	runAsUser 10001, runAsNonRoot true	Root containers can read /etc/shadow and modify host mounts
Read-only root FS	readOnlyRootFilesystem true	Prevents writing a web shell or modifying binaries
Dropped capabilities	capabilities drop ALL	Removes NET_RAW and SYS_ADMIN among others
No escalation	allowPrivilegeEscalation false	Blocks setuid binaries from gaining root
Seccomp	RuntimeDefault	Filters roughly 40 dangerous syscalls at the kernel
No API token	automountServiceAccountToken false	Pods do not call the Kubernetes API; a stolen token enables lateral movement
Memory limit	Set on every container	A leak is contained rather than evicting neighbours
No CPU limit	Deliberately omitted	CFS throttling adds tail latency even with spare node capacity; the HPA handles growth
Immutable tags	SHA or digest; chart rejects latest	A rollback to a mutable tag is not a rollback

7.2 Argo CD Sync Waves

Wave	Contents	Dependency
-2	Namespaces with Pod Security Standard labels	Everything lands inside them
-1	CRDs: external-secrets, prometheus-operator, KEDA	Charts creating those resources fail if the CRD is absent
0	ingress-nginx, cert-manager, ESO, kube-prometheus-stack, KEDA	Workloads depend on all of them
1	ClusterSecretStore, ClusterIssuer, KEDA TriggerAuthentication	Require the wave-0 controllers running
2	shift-api, whose PreSync hook migrates the schema	Schema must be current before consumers start
3	roster-worker, web	Depend on the API service and the migrated schema

7.3 Zero-Secret Identity Model

The trust chain, stated precisely because this is the part that fails most often and most opaquely:

- AKS projects a service account token into the pod with a one-hour TTL and a scoped audience.
- A federated identity credential on the managed identity matches the token's issuer (the cluster OIDC endpoint) and its subject, which must be exactly `system:serviceaccount:<namespace>:<serviceaccount>`.
- Azure returns an Entra access token, used for SQL, Service Bus and Key Vault.

If the service account name, the namespace, or the client-id annotation disagree by even one character, token exchange fails with AADSTS700213. Azure does not reveal which field mismatched — verifying all three sides of the triangle is the only reliable diagnosis.

8. Deployment Guide with Verified Evidence

This section is the reproducible build sequence. Each phase lists the exact commands and shows the verified result from the live deployment.

8.1 Phase 0 — Local Verification (no Azure cost)

Run every check locally before touching Azure. These cost nothing and catch the majority of defects while they are still cheap to fix.

#	Command	Expected Result
0.1	<code>./scripts/preflight.sh</code>	All checks pass — tooling, session, RBAC, providers, quota
0.2	<code>cd services/shift-api && py -m pip install -r requirements-dev.txt && py -m pytest</code>	17 passed, 82% coverage
0.3	<code>cd services/roster-worker && py -m pip install -r requirements-dev.txt && py -m pytest</code>	44 passed, 79% coverage
0.4	<code>cd services/web && npm ci && npm test && npx tsc -b</code>	13 passed, no type errors
0.5	<code>make lint && make audit</code>	Clean; pod security gate passes
0.6	<code>cd infra/envs/dev && checkov -d .</code>	0 failed

On Windows use `py -m pip` and `py -m pytest`. The bare `python` and `pip` commands are intercepted by App Execution Aliases and redirect to the Microsoft Store. See Section 9.1.

8.2 Phase 1 — Identity and State Backend

Two Entra ID security groups are created first: one for AKS cluster administration, one for Azure SQL administration. Their object IDs become Terraform inputs, and using groups rather than individual users means access is managed through group membership rather than by re-running Terraform.

2 groups found

☐	Name ↑	Object Id	Group type	Membershi
☐	SA ShiftBoard AKS Admins	7fe77681-43ce-422a-966c-84731c07c492	Security	Assigned
☐	SS ShiftBoard SQL Admins	8b253d16-004a-4e9d-bedd-ee1a2e46e6db	Security	Assigned

Figure 2. Entra ID security groups created for AKS and SQL administration. The object IDs are supplied to Terraform as `aks_admin_group_object_ids` and `sql_admin_group_object_id`.

The state backend is then bootstrapped. This is the one Terraform root module that uses local state, because state must exist somewhere before Terraform can manage state.

```
My Pc@DESKTOP-HNESNGA MINGW64 ~/Desktop/Projects/P4-Shiftboard/infra/bootstrap
$ terraform apply -auto-approve

Outputs:

backend_config = {
  "container_name" = "tfstate"
  "resource_group_name" = "rg-shiftboard-tfstate"
  "storage_account_name" = "stshiftboardtfstate01"
}
```

Figure 3. terraform apply in infra/bootstrap. The backend_config output supplies the resource group, storage account and container name used by every environment backend block.

tfstate

Container

Search

+ Add Directory ↑ Upload ↻ Refresh | 🗑 Delete 📄 Copy 📄 Paste 🏷 Rename 🔗 Acquire I

tfstate > dev

Authentication method: Microsoft Entra user account ([Switch to Access key](#))

Search blobs by prefix (case-sensitive)

Showing all 1 items

☐	Name	Last modified	Access tier	Blob type
☐	📁 [.]			
☐	📄 shiftboard.tfstate	12/09/2026, 19:18:10	Hot (Inferred)	Block blob

Figure 4. The rg-shiftboard-tfstate resource group containing the state storage account.

stshiftboardtfstate01 | Containers

Storage account

Search

+ Add container ↑ Upload ↻ Refresh | 🗑 Delete 🔒 Change access leve

Search containers by prefix

Showing all 2 items

☐	Name	Last modified	Anonymous acc
☐	📁 \$logs	12/09/2026, 14:55:41	Private
☐	📁 tfstate	12/09/2026, 14:55:53	Private

Figure 5. Blob containers inside stshiftboardtfstate01. Shared access keys are disabled; the backend authenticates with Entra ID via use_azuread_auth.

8.3 Phase 2 — Infrastructure Provisioning

With the backend in place, the development environment is applied. Note the `MSYS_NO_PATHCONV=1` prefix — without it Git Bash rewrites Azure resource paths as Windows directory paths and the apply fails.

```
My Pc@DESKTOP-HNESNGA MINGW64 ~/Desktop/Projects/P4-Shiftboard/infra/envs/dev
$ MSYS_NO_PATHCONV=1 terraform apply -auto-approve

Apply complete! Resources: 1 added, 2 changed, 0 destroyed.

Outputs:

acr_login_server = "crshiftboarddevar01.azurecr.io"
aks_cluster_name = "aks-shiftboard-dev"
helm_values = {
  "acrLoginServer" = "crshiftboarddevar01.azurecr.io"
  "environment" = "dev"
  "externalSecretsClientId" = "d5c1b1ea-6f9a-467d-b246-d80e4ce1670e"
  "keyVaultName" = "kv-shiftboarddevar01"
  "keyVaultUri" = "https://kv-shiftboarddevar01.vault.azure.net/"
  "rosterWorkerClientId" = "c5fc9b97-1735-422e-85cd-d820261de041"
  "serviceBusNamespace" = "sb-shiftboard-dev-ar01"
  "serviceBusQueue" = "roster-events"
  "shiftApiClientId" = "cb4f000d-1925-42ef-8f6c-de64c31bbf40"
  "sqlDatabaseName" = "shiftboard"
  "sqlServerFqdn" = "sql-shiftboard-dev-ar01.database.windows.net"
  "tenantId" = "7f5f4f67-d486-4c43-a0db-199d0b40e057"
}
key_vault_name = "kv-shiftboarddevar01"
key_vault_uri = "https://kv-shiftboarddevar01.vault.azure.net/"
log_analytics_workspace_id = "/subscriptions/64c8dabf-3f71-4501-969f-bfbdd80b883d/resourceGroups/rg-shiftboard-dev/providers/Microsoft.OperationalInsights/workspaces/log-shiftboard-dev"
```

Figure 6. terraform apply for the dev environment. The `helm_values` output block contains every value the Helm charts need — managed identity client IDs, the SQL FQDN, the Service Bus namespace and the Key Vault URI — so no value is ever hand-copied into a values file.

The apply produces three resource groups. The AKS node resource group is created and managed by Azure itself rather than by Terraform.

☐	Name ↑
☐	 NetworkWatcherRG
☐	 rg-shiftboard-dev
☐	 rg-shiftboard-dev-aks-nodes
☐	 rg-shiftboard-tfstate

Figure 7. Resource groups after apply: rg-shiftboard-dev holds the platform, rg-shiftboard-dev-aks-nodes is Azure-managed, rg-shiftboard-tfstate holds Terraform state.

Twenty-three resources were provisioned into rg-shiftboard-dev. The three pages below show the complete inventory.

 **rg-shiftboard-dev**
Resource group
Export resource groups using Bicep or Terraform
Generate Bicep code to duplicate this resc

Filter for any field...
Type equals all
Location equals all
+ Add filter

- Overview
- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

☐	 ag-shiftboard-dev-oncall	...	Action group
☐	 aks-shiftboard-dev	...	Kubernetes service
☐	 alert-shiftboard-dev-dlq-depth	...	Metric alert rule
☐	 alert-shiftboard-dev-sql-cpu	...	Metric alert rule
☐	 alert-shiftboard-dev-sql-storage	...	Metric alert rule
☐	 crshiftboarddevar01	...	Container registry
☐	 id-shiftboard-dev-aks-cp	...	Managed Identity
☐	 id-shiftboard-dev-external-secrets	...	Managed Identity
☐	 id-shiftboard-dev-roster-worker	...	Managed Identity
☐	 id-shiftboard-dev-shift-api	...	Managed Identity

Showing 1 - 10 of 23. Display count: auto
< 1 2 3 >

Add or remove favorites by pressing Ctrl+Shift+F

Figure 8. Resources page 1 — action group, AKS cluster, three metric alert rules (DLQ depth, SQL CPU, SQL storage), container registry, and four user-assigned managed identities including one per workload.

rg-shiftboard-dev Resource group

Export resource groups using Bicep or Terraform Generate Bicep code to duplicate this resou

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

Create Manage view Delete resource group Refresh Export to CSV Open

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type
kv-shiftboarddevar01	Key vault
log-shiftboard-dev	Log Analytics workspace
nsg-shiftboard-dev-aks	Network security group
nsg-shiftboard-dev-pe	Network security group
pe-sql-shiftboard-dev-ar01	Private endpoint
pe-sql-shiftboard-dev-ar01.nic.f145e046-0d7a-4b72-bf51-17b6bc51	Network Interface
privatelink.database.windows.net	Private DNS zone
privatelink.servicebus.windows.net	Private DNS zone
privatelink.vaultcore.azure.net	Private DNS zone

Showing 11 - 20 of 23. Display count: auto

Figure 9. Resources page 2 — Key Vault, Log Analytics workspace, two network security groups (AKS subnet and private endpoint subnet), the SQL private endpoint with its NIC, and three private DNS zones for database, Service Bus and Key Vault.

rg-shiftboard-dev Resource group

Export resource groups using Bicep or Terraform Generate Bicep code to duplicate this resou

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring

Create Manage view Delete resource group Refresh Export to CSV Open

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type
shiftboard (sql-shiftboard-dev-ar01/shiftboard)	SQL database
sql-shiftboard-dev-ar01	SQL server
vnet-shiftboard-dev	Virtual network

Figure 10. Resources page 3 — the SQL server, the shiftboard database, and the virtual network.

 **rg-shiftboard-tfstate**   

Resource group

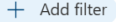
Search    Create  Manage view  Delete resource group  Refresh  Export to CSV  Open query | 

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management

Essentials

Resources Recommendations

 Filter for any field...  Type equals all  Location equals all  Add filter

☐	Name ↑	Type
☐	 stshiftboardtfstate01	 Storage account

Figure 11. The Azure-managed node resource group: two VM scale sets (system and application pools), the cluster load balancer, the agent pool identity, and the Key Vault secrets provider and Azure Policy add-on identities.

☐		Name	Tag	Image ID	Created	Size	Actions
☐		crshiftboarddevar01.az	1.0.0	3c3c37866d0e	20 hours ago	433.17 MB	   
☐		shift-api	1.0.0	3c3c37866d0e	20 hours ago	433.17 MB	   
☐		crshiftboarddevar01.az	1.0.0	3722f8ca1bab	20 hours ago	365.67 MB	   
☐		roster-worker	1.0.0	3722f8ca1bab	20 hours ago	365.67 MB	   
☐		crshiftboarddevar01.az	1.0.0	3640d077f891	19 hours ago	74.37 MB	   
☐		web	1.0.0	3640d077f891	19 hours ago	74.37 MB	   

Figure 12. The dev state file written to the tfstate container. Authentication shown as Microsoft Entra user account, confirming the backend is not using shared access keys.

8.4 Phase 3 — Cluster Access and Database Grants

Cluster credentials are fetched with `--overwrite-existing` to force a clean context, then node readiness is confirmed.

```
My Pc@DESKTOP-HNESNGA MINGW64 ~/Desktop/Projects/P4-Shiftboard/infra/envs/dev
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
aks-app-12971302-vmss000000	Ready	<none>	19m	v1.34.0
aks-system-34709049-vmss000000	Ready	<none>	49m	v1.34.10

Figure 13. Both node pools Ready. The system pool carries the `CriticalAddonsOnly` taint so application workloads land on the application pool.

Database users are then created. Terraform cannot do this: contained database users live inside the database rather than in the Azure control plane, so they require a T-SQL connection. Because the SQL server has no public endpoint, the script runs from a pod inside the VNet using the `sqlcmd` container image.

shift-api receives `db_datareader`, `db_datawriter` and `db_ddladmin` — the last because Alembic migrations require DDL. roster-worker receives only `db_datareader` and `db_datawriter`; it never migrates the schema, so it must not be able to alter it.

8.5 Phase 4 — Container Images

Three multi-stage images are built. Each has a builder stage carrying compilers and a runtime stage that does not, runs as UID 10001, and uses `tini` as PID 1 so `SIGTERM` reaches the application and connections drain cleanly.

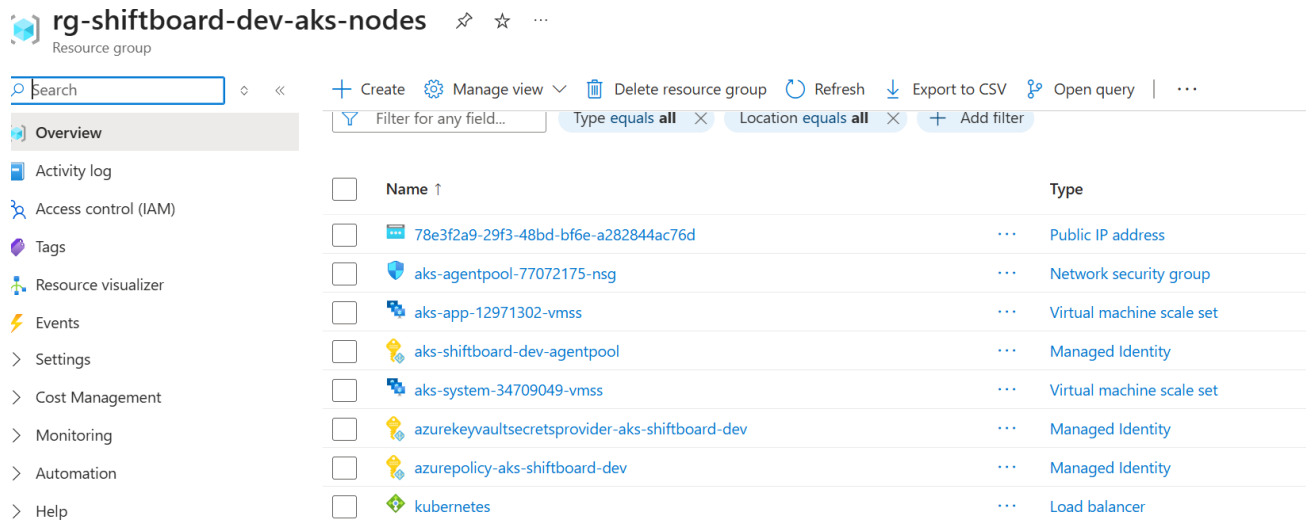


Figure 14. The three images built locally and tagged for the registry. `shift-api` and `roster-worker` carry the Microsoft ODBC driver for Azure SQL; the web image is far smaller because it contains only static assets and `nginx`.

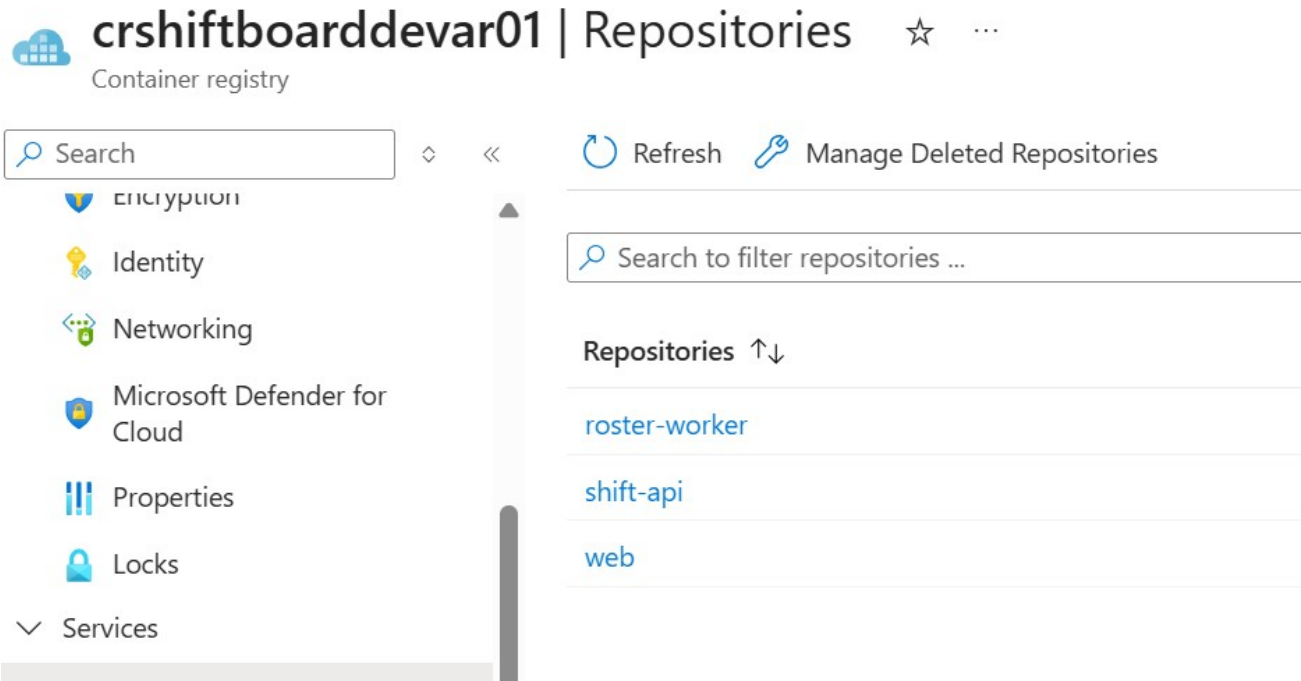


Figure 15. The three repositories in Azure Container Registry after push. The admin user is disabled — the cluster kubelet pulls using its managed identity with the AcrPull role, so no imagePullSecret exists.

8.6 Phase 5 to 7 — GitOps Deployment and Verification

Argo CD is installed with Helm — the one component that cannot be managed by Argo CD itself. The root application is then applied, and from that point every change is a Git commit.

aks-shiftboard-dev | Namespaces ☆ ...
Kubernetes service

Search ◇ << + Create ▾ Delete Refresh Show labels → Convert to Managed Manage

Access control (IAM)
Tags
Monitor (Insights)
Diagnose and solve problems
Microsoft Defender for Cloud
Cost analysis
Resource visualizer
Kubernetes resources

Namespaces
Workloads
Services and ingresses
Network policies

Namespace name : All Type equals All Add label filter

☐	Name	Status	Age
☐	kube-system	✓ Active	17 hours
☐	default	✓ Active	17 hours
☐	kube-node-lease	✓ Active	17 hours
☐	kube-public	✓ Active	17 hours
☐	calico-system	✓ Active	17 hours
☐	tigera-operator	✓ Active	17 hours
☐	gatekeeper-system	✓ Active	17 hours
☐	shiftboard	✓ Active	15 hours

Add or remove favorites by pressing **Ctrl+Shift+F**

Figure 16. Cluster namespaces. *calico-system* provides NetworkPolicy enforcement, *gatekeeper-system* provides Azure Policy admission control, and *shiftboard* holds the application workloads under the restricted Pod Security Standard.

aks-shiftboard-dev | Workloads ☆ ...
Kubernetes service

Search

Cost analysis
Resource visualizer
Kubernetes resources
Namespaces
Workloads
Services and ingresses
Network policies
Storage
Configuration
Custom resources
Events
Run command

+ Create Scale Delete Refresh Show labels Give feedback

☐	calico-kube-controllers	calico-system	✓ 1/1
☐	gatekeeper-controller	gatekeeper-system	✓ 2/2
☐	gatekeeper-audit	gatekeeper-system	✓ 1/1
☐	azure-policy-webhook	kube-system	✓ 1/1
☐	azure-policy	kube-system	✓ 1/1
☐	metrics-server	kube-system	✓ 2/2
☐	konnectivity-agent-autosca	kube-system	✓ 1/1
☐	konnectivity-agent	kube-system	✓ 2/2
☐	azure-wi-webhook-controll	kube-system	✓ 2/2
☐	tigera-operator	tigera-operator	✓ 1/1
☐	coredns-autoscaler	kube-system	✓ 1/1
☐	coredns	kube-system	✓ 2/2

Figure 17. Platform workloads healthy: Calico controllers, Gatekeeper controller and audit, Azure Policy webhook, metrics-server, konnectivity agents, and azure-wi-webhook-controller — the last being the component that injects the projected service account token used by workload identity.

aks-shiftboard-dev | Services and ingresses ☆ ...
Kubernetes service

Search

Cost analysis
Resource visualizer
Kubernetes resources
Namespaces
Workloads
Services and ingresses
Network policies
Storage
Configuration
Custom resources
Events
Run command

+ Create Delete Refresh Show labels Give feedback

☐	kubernetes	default	✓ Ok	ClusterIP
☐	kube-dns	kube-system	✓ Ok	ClusterIP
☐	azure-wi-webhook-web...	kube-system	✓ Ok	ClusterIP
☐	metrics-server	kube-system	✓ Ok	ClusterIP
☐	azure-policy-webhook-s...	kube-system	✓ Ok	ClusterIP
☐	gatekeeper-webhook-se...	gatekeeper-system	✓ Ok	ClusterIP
☐	calico-typha	calico-system	✓ Ok	ClusterIP
☐	calico-kube-controllers...	calico-system	✓ Ok	ClusterIP
☐	shift-api	shiftboard	✓ Ok	ClusterIP
☐	roster-worker	shiftboard	✓ Ok	ClusterIP
☐	web	shiftboard	✓ Ok	ClusterIP

Figure 18. All three application services running in the shiftboard namespace as ClusterIP. None is exposed as a LoadBalancer — external traffic arrives only through the ingress path.

The single most important verification is not that pods show Running. It is that `roster_worker_consumer_up` equals 1. A worker pod can be perfectly healthy while its Service Bus receive loop is failing silently, in which case the queue grows and nothing is processed.

9. Azure Student Subscription: Issues and Resolutions

Azure for Students imposes hard constraints that do not exist on Pay-As-You-Go or Enterprise subscriptions, and quota increase requests are denied for student accounts. **Read this section before running the first apply** — several fixes must be made to the Terraform code beforehand, and discovering them mid-apply leaves a partially provisioned environment that is harder to clean up than starting correctly.

Every issue below was encountered and resolved during the real deployment documented in Section 8.

9.1 Windows Development Environment

ID	Issue	Root Cause	Resolution
WIN-1	Python opens the Microsoft Store	Windows App Execution Aliases intercept the words python and pip	Disable the aliases in Settings > Apps > Advanced app settings, or use <code>py -m pip</code> and <code>py -m pytest</code>
WIN-2	Git Bash corrupts Azure URLs — ://azure.com becomes C:/Program Files/Git/azure.com	The MSYS layer treats forward slashes as Windows paths and rewrites them before the command runs	Prefix commands with <code>MSYS_NO_PATHCONV=1</code> , or use PowerShell for Azure CLI and Terraform
WIN-3	PowerShell refuses to load the SqlServer module	Host execution policy blocks third-party modules	<code>Set-ExecutionPolicy Bypass -Scope Process</code> — affects only the current terminal session
WIN-4	kubelogin.exe not found after installation	az aks install-cli writes to a directory absent from the terminal's PATH, which was fixed at launch	Append the install directory to PATH in the current session
WIN-5	Copy-pasted image references are truncated	Git Bash clipboard handling reinterprets strings containing ://	Type registry paths manually or use PowerShell for those commands

9.2 Subscription Resource Constraints

ID	Issue	Root Cause	Terraform Fix
SUB-1	EncryptionAtHost unavailable — apply fails on feature registration	Host encryption needs the Microsoft.Compute/Encryption AtHost feature flag, unavailable on student subscriptions	Set <code>host_encryption_enabled = false</code> on both node pools in <code>infra/modules/aks/main.tf</code>
SUB-2	QuotaExceeded on Total Regional Cores	Student subscriptions are hard-limited to 4 vCPUs per region. Two D2s_v5 pools exceed that once autoscaling engages.	Use <code>Standard_B2s</code> (2 vCPU, burstable) and set <code>max_count = 1</code> on both pools — exactly 4 vCPUs total

ID	Issue	Root Cause	Terraform Fix
SUB-3	Availability zone not supported in region	Several student-accessible regions, including East Asia, do not offer availability zones	Set <code>availability_zones = []</code> so Azure places nodes automatically
SUB-4	AADSTS50076 on az login	Student tenants enable Security Defaults, which enforce MFA and can block device-code flows	Use <code>az login --tenant <tenant-id></code> to force a browser-based MFA flow
SUB-5	401 Unauthorized midway through a long apply	AKS provisioning takes 10-15 minutes and the CLI access token can expire during it	Re-authenticate immediately before the apply. Terraform is idempotent, so a failed run can simply be repeated

9.3 Terraform Code Corrections

ID	Issue	Root Cause	Resolution
TF-1	var.kubernetes_version undeclared	The module declares the variable but the composition passes a hardcoded string	Confirm the variable exists with a default and pass it explicitly. Check supported versions with <code>az aks get-versions</code>
TF-2	for_each depends on an unknown value on the <code>AcrPull</code> role	The role assignment depends on the AKS kubelet identity, which does not exist at plan time	Move the role assignment out of the ACR module into the environment composition, breaking the circular dependency
TF-3	Duplicate output declaration	The same output declared in both <code>main.tf</code> and <code>outputs.tf</code>	Remove the duplicate and re-run <code>terraform validate</code>
TF-4	Placeholder tenant ID causes 401 on Key Vault	<code>terraform.tfvars</code> still contained the <code>11111111-...</code> example value	Replace with the real value from az account <code>show --query tenantId -o tsv</code>

9.4 Kubernetes and Runtime Issues

ID	Issue	Root Cause	Resolution
K8S-1	kubectl targets a cluster that no longer exists	A previous project left its context as the default in <code>kubeconfig</code>	<code>az aks get-credentials ... --overwrite-existing</code> , then confirm with <code>kubectl config current-context</code>
K8S-2	Unable to find exec auth plugin azure	Entra-only AKS requires the <code>kubelogin</code> credential plugin, which is not bundled with <code>kubectl</code>	<code>az aks install-cli</code> , then add the install directory to <code>PATH</code>
K8S-3	Pods cannot pull from Docker Hub	Nodes in the private subnet have no NAT gateway for outbound internet access	Pre-push required images to ACR, or run one-off tasks from the workstation instead of from a pod
K8S-4	ODBC error FA005 — shift-api in CrashLoopBackOff	Empty default strings for <code>db_user</code> and <code>db_password</code> caused ODBC Driver 18 to	Explicitly null the fallback credentials in the values overlay so only the workload identity token channel remains open

ID	Issue	Root Cause	Resolution
		attempt SQL authentication at the same time as workload identity. Two authentication methods competed for the same connection.	
K8S-5	az sql extension unavailable	Some Azure CLI versions omit the extension, and Microsoft removed certain query actions	Install the db-up extension, or run the SQL script from a pod using the sqlcmd container image

9.5 Pre-Apply Checklist for Student Subscriptions

#	Check	Verify With	If Wrong
1	Tenant ID is real, not the placeholder	grep '11111111' terraform.tfvars returns nothing	az account show --query tenantId -o tsv
2	name_suffix is set and unique to you	grep name_suffix terraform.tfvars	ACR, Key Vault, SQL and Service Bus names are globally unique across Azure
3	host_encryption_enabled is false	grep host_encryption infra/modules/aks/main.tf	Set false on both node pools
4	VM size is Standard_B2s	grep vm_size infra/envs/dev/main.tf	Replace Standard_D2s_v5
5	max_count is 1 on both pools	grep max_count infra/envs/dev/main.tf	4 vCPUs total is the student ceiling
6	availability_zones is empty	grep availability_zones infra/envs/dev/main.tf	Change ["1"] to []
7	Signed in with explicit tenant	az account show	az login --tenant <tenant-id>
8	Region offers Standard_B2s	az vm list-sizes -l <region>	Try eastus, westeurope or southeastasia
9	AcrPull role is not inside a dynamic loop	Inspect modules/acr/main.tf	Move it to envs/dev/main.tf
10	MSYS_NO_PATHCONV is set in Git Bash	echo \$MSYS_NO_PATHCONV	export MSYS_NO_PATHCONV=1 in the shell profile

10. Testing and Verification

10.1 Automated Test Suites

All suites run against in-memory SQLite and fake Service Bus objects, so they require no cloud credentials and complete in under three seconds.

Suite	Tests	Coverage	Key Areas
shift-api	17	82% (gate 80%)	CRUD, optimistic concurrency races, duplicate site 409, naive timestamp rejection, health probes, metrics
roster-worker rules	15	—	Overlap, touching shifts not overlapping, rest gaps before and after, zero-rest policy, weekly hours at limit, cross-week exclusion
roster-worker handler	13	—	Idempotency ledger, rescan replaces rather than appends, cancellation clears conflicts, unknown worker dead-lettered
roster-worker consumer	6	—	Complete, abandon and dead-letter settlement matrix, drain loop, SIGTERM mid-batch, receive error survival
roster-worker timezone	4	—	Regression coverage for the naive/aware datetime boundary
web	13	—	Timezone-correct rendering, day bucketing, duration maths, coverage meter states, claim interaction, conflict strip
Total	68 + 6	79-82%	Full suite green

10.2 Defects Caught Before Deployment

Timezone boundary defect. SQLAlchemy's DateTime(timezone=True) maps to SQL Server datetime2, which stores no UTC offset. Values written as timezone-aware are read back naive, and the first rule evaluation on reloaded data raised TypeError: can't compare offset-naive and offset-aware datetimes. Resolved with a normalising function at the rule-engine boundary plus four regression tests. This class of defect only appears once data round-trips through a real database, which is exactly why the test suite exercises persistence rather than mocking it.

nginx configuration defect. The upstream host placeholder was written into conf.d/default.conf, where nothing performs substitution. Moved to /etc/nginx/templates/ so the official entrypoint runs envsubst at container start.

10.3 Static Analysis and Security Gates

Gate	Result	Enforcement
Checkov — production baseline	59 passed, 0 failed, 18 documented inline skips	Hard fail; no environment-level exemptions permitted
Checkov — development	57 passed, 0 failed	Two scoped exemptions in envs/dev/.checkov.yaml, each with written justification
Ruff (Python lint)	Zero findings	Hard fail
TypeScript compilation	Clean	Hard fail

Gate	Result	Enforcement
Helm lint	3 of 3 charts pass	Hard fail
Helm template render	24 manifests, all valid YAML	Hard fail
Pod security audit gate	4 of 4 containers pass	Hard fail; gate itself verified by injecting violations and confirming exit code 1
Chart value guards	4 of 4 fire correctly	Missing tag, missing repository, auth without tenant, digest precedence over tag
Trivy image scan	Gates every push	Hard fail on HIGH or CRITICAL

The Checkov exemptions are scoped to the development environment only. Suppressing them inline in the shared modules would have exempted production from the same checks — production is scanned against the full, unmodified baseline.

11. Observability

Metric	Type	Significance
shiftboard_shifts_created_total	Counter	Shift creation rate by site
shiftboard_shift_claims_total	Counter	Claim outcomes: accepted, conflict, not_found, forbidden
shiftboard_event_publish_failures_total	Counter	Events lost to Service Bus outages — any value above zero warrants investigation
shiftboard_open_shifts	Gauge	Currently unfilled shifts by site
shiftboard_db_query_seconds	Histogram	Domain query latency; P99 above 250 ms is a scaling signal
roster_worker_events_processed_total	Counter	Consumer throughput and dead-letter rate
roster_worker_conflicts_detected_total	Counter	Conflicts found, segmented by rule
roster_worker_consumer_up	Gauge	The single most important operational metric — 1 connected, 0 receive loop failing
roster_worker_event_lag_seconds	Histogram	Delay between event occurrence and processing; drives the freshness SLO

Two alerting layers cover different failure domains. Azure metric alerts fire on platform signals that never reach Prometheus — dead-letter queue depth, SQL CPU and SQL storage.

PrometheusRules fire on application SLOs. Alertmanager groups by alert name and service so one broken service does not bury alerts from another, and an inhibition rule suppresses latency alerts when the service is entirely down: one page per incident, not five.

12. Cost Management

Cost is treated as an engineering constraint rather than an afterthought, which matters especially on a student subscription with a fixed credit balance.

- **Consumption budget** with alerts at 80% actual and 100% forecast spend, provisioned by Terraform.
- **Burstable B2s nodes** with max_count 1 per pool — required by the 4 vCPU student quota and substantially cheaper than the D-series default.
- **Serverless Azure SQL** which auto-pauses after one hour idle, so an unused database costs storage only.
- **Log Analytics daily ingestion cap** so a log storm cannot produce an unbounded bill.
- **ACR untagged manifest retention** of 7 days in development, bounding registry storage growth.
- **No Front Door in development** — the WAF alone costs roughly USD 35 per month and adds no learning value after the first build.

Environment	Estimated Monthly	Principal Drivers
Development (student configuration)	USD 60-110	AKS node hours, SQL storage, Service Bus Standard, Log Analytics
Production reference configuration	USD 800-1,300	Three-zone on-demand nodes, provisioned SQL, ACR Premium, Service Bus Premium, Front Door and WAF

An idle AKS cluster still bills. Tear the environment down between working sessions with `make tf-destroy ENV=dev`; it rebuilds in about 25 minutes. Always delete the Argo CD root application first and wait for the load balancer to disappear, otherwise an orphaned public IP continues to incur charges.

13. Troubleshooting Reference

Symptom	Root Cause	Resolution
AADSTS700213 or CredentialUnavailableError	Federated credential subject mismatch	Service account annotation, pod label azure.workload.identity/use, and the federated subject must agree exactly
Pods stuck in Pending	Node taint not tolerated, or vCPU quota exhausted	Add the toleration, or reduce node size and count for a student subscription
Login failed for user '<token-identified principal>'	Database grants were never applied	Run the grant script from inside the VNet
nginx exits immediately on start	The upstream host is wrong or the service does not yet exist	shiftApiHost must match the actual service name produced by the Helm release
terraform destroy hangs on the VNet	A load balancer or public IP is still attached	Delete the Argo CD root application first and wait for removal
Every network call inside pods times out	NetworkPolicy has no DNS egress rule	Allow egress to kube-system kube-dns on port 53, UDP and TCP
Argo CD reports app path outside root	A value file path escapes the chart directory	Use the multi-source \$values reference pattern
Metrics absent from Grafana	ServiceMonitor label does not match the Prometheus selector	Every ServiceMonitor needs release: kube-prometheus-stack
ODBC FA005 on shift-api	Empty credential defaults collide with the workload identity token	Null the fallback credentials explicitly in the values overlay

14. Scope Boundaries and Planned Work

Stated explicitly so a reviewer can see these are decisions rather than omissions.

Not Included	Rationale	Planned
Progressive delivery with automated rollback	Rolling updates are the correct starting point; canary analysis needs metrics flowing first	Next major iteration
Image signing, SBOM, Kyverno admission	Trivy gates the pipeline today; admission-time verification is the next security layer	Next major iteration
Multi-region and disaster recovery	Single region is honest. A DR claim is worthless unless RTO and RPO have been measured in a real failover drill	Future
Disk encryption set	Requires a Key Vault key with its own rotation and RBAC lifecycle — belongs to a platform landing zone, not an application cluster	Future
Azure DevOps pipeline YAML	The manual path is fully functional and documented; pipeline automation is the highest-priority remaining item	Immediate next step
PrometheusRules and Grafana dashboards	Metrics are exposed and scraped; alert rules and dashboards are configuration on top of working telemetry	Immediate next step

15. Competencies Demonstrated

Area	Evidence in This Project
Cloud architecture	Three-tier event-driven system with asynchronous processing, private networking and managed PaaS services, deployed and verified live
Infrastructure as Code	10 Terraform modules, two environment compositions, remote state with Entra authentication and no shared access keys
Kubernetes engineering	AKS with CNI Overlay, Calico NetworkPolicy, HPA, PDB, topology spread, PreSync migration hooks, restricted Pod Security Standard
Helm development	Three charts with fail-fast value guards, conditional KEDA and HPA, environment overlays, automated manifest auditing
GitOps	Argo CD app-of-apps with six ordered sync waves, multi-source value files, selfHeal and prune, sync-failure notification
Security engineering	Zero-secret architecture, Entra-only SQL, private endpoints, microsegmentation, read-only root filesystems, dropped capabilities
Observability	Nine custom Prometheus metrics, structured JSON logging, control-plane audit collection, two-layer alerting with inhibition rules
Event-driven design	PEEK_LOCK settlement matrix, dead-letter handling, idempotency ledger, fail-open publishing, KEDA queue-depth scaling

Area	Evidence in This Project
Automated testing	74 unit tests including a full message settlement matrix, a timezone regression suite, and optimistic concurrency race coverage
Cost engineering	Consumption budget, burstable nodes within a 4 vCPU quota, serverless database, ingestion caps, documented cost trade-offs
Production troubleshooting	18 deployment errors diagnosed and resolved with documented root causes, plus two application defects caught by tests

— End of Document —